LLMs • RAG • AGENTS • MULTIMODAL • FINE-TUNING • SERVING

Cracking the Generative AI Engineering Interview 2026

The Essential Cheat Sheet for LLMs, RAG, AI Agents, Multimodal AI, Fine-Tuning, and Production AI Systems

AI ENGINEERING INSIDER • aiengineeringinsider.com



Cracking the Generative AI Engineering Interview 2026
The Essential Cheat Sheet for LLMs, RAG, AI Agents, Multimodal AI, Fine-Tuning, and Production AI Systems

Copyright © 2026 AI Engineering Insider. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the publisher, except in the case of brief quotations embodied in reviews and certain other noncommercial uses permitted by copyright law.

First edition, 2026. Published by AI Engineering Insider.
Web: aiengineeringinsider.com Newsletter: aiengineeringinsider.substack.com/subscribe

Disclaimer. The information in this book is provided on an "as is" basis, without warranty of any kind. Product names, logos, and brands—including OpenAI, Anthropic, Google, Meta, Llama, DeepSeek, Qwen, Mistral, NVIDIA, vLLM, SGLang, TensorRT-LLM, llama.cpp, Ollama, LangChain, LangGraph, CrewAI, AutoGen, Hugging Face, Pinecone, Qdrant, Weaviate, Milvus, and PostgreSQL—are the property of their respective owners and are used for identification purposes only; their use does not imply endorsement. This book is an independent educational work and is not affiliated with or endorsed by any model provider, framework maintainer, or standards body named within.

On figures that change. Model names, context limits, prices, benchmark scores, and framework capabilities change frequently. Every specific number in this book is a snapshot intended to support reasoning about *trade-offs*. Verify each figure against the primary source before relying on it in a design document or a purchasing decision. The publisher assumes no responsibility for errors or omissions, or for damages resulting from the use of the information contained herein.

On the security material. Chapters 3, 6, 9, and 10 describe prompt injection, jailbreak, and adversarial testing techniques at the level required to defend against them. The material is defensive in orientation and is intended solely for testing and hardening systems you are authorised to test.

Preface

Subscribe → aiengineeringinsider.substack.com/subscribe

A language model reads a token and predicts a token. Everything else in a generative AI system — the tokenizer, the retriever, the reranker, the KV cache, the agent loop, the guardrail — exists to decide *which* tokens it reads, and to make the reading cheap enough that the product survives contact with traffic.

That is not how generative AI is usually taught, and it is not how most interviews are prepared for. The common preparation is a vocabulary exercise: know what RAG stands for, know that LoRA is parameter-efficient, know that agents use tools. That vocabulary gets you through a screening call and collapses in the second round, because the questions that actually distinguish candidates are not definitional. They are: *how large is the KV cache for this model at this context, and what does that mean for your batch size? Your retrieval returns semantically similar but factually wrong passages — is a better embedding model the fix? Your agent works in testing and loops in production — walk me through the debugging.*

Those questions have real answers, and the answers are engineering rather than trivia. This book is organised around producing them. Each chapter builds the mental model for one layer of the stack, states the trade-offs where decisions actually branch, and then works through ten interview questions with answers at the depth a senior interview expects — system design, production operations, debugging, technical mechanism, and case studies drawn from the failure modes that recur across real deployments.

The perspective throughout is a system designer's. Research questions ask which method is better. Production questions ask which method is better *at this scale, on this traffic mix, under this latency SLO, with this failure tolerance* — and the answer changes with all four. Grouped-query attention is not better than multi-head attention; it is the configuration that makes a 70B model economically servable. Retrieval-augmented generation is not better than a long context window; it is what you use when the corpus updates, when answers need citations, and when a customer can ask you to delete their data. Chapter after chapter, the argument is not that one approach wins, but that you should be able to say which one wins for you, and show the number.

More practitioner deep dives: https://aiengineeringinsider.substack.com/subscribe

AI Engineering Insider
2026

About This Book

Subscribe → aiengineeringinsider.substack.com/subscribe

This book is a system-design-level interview guide to generative AI: how text becomes tokens, how tokens become answers, how those answers are grounded, served, secured, evaluated, and paid for. It is written for AI engineers, machine learning engineers, inference and ML systems engineers, agentic-AI and forward-deployed engineers, software engineers moving into AI, and AI architects who are past the tutorials and need the patterns, trade-offs, failure modes, and runnable code that show up in production and in senior interviews.

The Design Contract

Principle	What it means in practice
Count the tokens	Every design decision has a token price. If you cannot state what one request costs, you cannot reason about latency, context, or bill.
Memory, not compute	After 2020 the binding constraint on language models became memory bandwidth. GQA, MoE, FlashAttention, and quantization are all bandwidth optimisations wearing different clothes.
Prefill and decode are different systems	One is compute-bound, the other bandwidth-bound. They want different batch sizes, different optimisations, increasingly different hardware. Half of all performance work optimises the wrong one.
Attribute before optimising	Retrieval error and generation error look identical from outside and have opposite fixes. So do prefill latency and queue latency. Decompose first, tune second.
Structure beats persuasion	Constrained decoding over "please return JSON." A policy engine over "do not exceed the limit." Structural mechanisms survive model upgrades; tuned wording does not.
Authority lives outside the model	Prompt injection is unsolved. Assume the prompt layer is bypassed and ask what the attacker gains. If it is more than a wrong sentence, the control is in the wrong layer.
Least autonomy that works	Every step up the agent ladder multiplies cost variance, failure modes, and security surface. Buy only the autonomy the problem requires.
Measure, do not assume	Cache hit rate, acceptance rate, recall@k, judge agreement, cost per successful task: all measurable, all routinely guessed, all routinely wrong.

How This Volume Is Organised

Ch.	Title	Main focus
1	Foundations of Generative AI	Discriminative versus generative modelling and the chain-rule factorisation; foundation-model families; self-attention, FFN, residuals, and normalisation placement; tokenization algorithms and the real cost of non-English text; embedding types and what cosine similarity cannot represent; scaling laws, Chinchilla optimality versus total cost of ownership, and emergent abilities.
2	Transformer and LLM Architecture	The modern decoder block and residual stream; MHA, MQA, GQA, and MLA as KV-cache economics; RoPE, ALiBi, and context extension; the KV-cache sizing formula and PagedAttention; Mixture-of-Experts and what it actually trades; FlashAttention and the roofline view of prefill versus decode.
3	Prompt Engineering	Prompt anatomy and the trust boundary; system-prompt design and instruction dilution; zero-shot, few-shot, and dynamic example selection; chain-of-thought and when it hurts; structured output and constrained decoding; prompt optimisation as a measured loop; injection, jailbreaks, leakage, and layered defence.
4	LLM Inference and Serving	The generation pipeline end to end; sampling methods and their production settings; continuous batching and chunked prefill; prefix caching as the highest-leverage optimisation; speculative decoding and where it backfires; vLLM, SGLang, TensorRT-LLM, and llama.cpp; serving topology, admission control, and autoscaling signals.
5	Retrieval-Augmented Generation	RAG against fine-tuning and long context; parsing and the failures that cap quality; chunking strategies and parent–child retrieval; dense, sparse, and hybrid retrieval with reciprocal rank fusion; vector stores and ANN index trade-offs; bi-encoders versus cross-encoders; query transformation; component-wise evaluation and error attribution.
6	AI Agents	The autonomy spectrum and why reliability compounds multiplicatively; agent components and tool design; memory types, provenance, and poisoning; context management and the structured scratchpad; multi-agent patterns and when they are justified; human-in-the-loop by reversibility; trajectory evaluation and cost per successful task.
7	Multimodal AI	The token cost of every modality; vision-language architecture and the connector's accuracy cliff; dynamic tiling and resolution as capability; OCR versus VLM extraction and silent transcription hallucination; ASR, TTS, and the voice-latency budget; diffusion and controllability; multimodal RAG and video sampling policy.
8	Fine-Tuning and Model Adaptation	Form versus fact as the decision criterion; SFT mechanics and loss masking; LoRA mathematics, target modules, and rank selection; QLoRA and the memory ladder; RLHF, DPO, IPO, KTO, and ORPO; dataset quality as the dominant variable; synthetic data and model collapse; catastrophic forgetting and capability gating.
9	Production Generative AI Systems	The reference architecture; API design for an expensive non-deterministic dependency; layered guardrails with explicit failure policy; the caching hierarchy and semantic-cache risk; token-based rate limiting; observability and cost attribution; Kubernetes deployment, cold starts, and autoscaling on the right signals.

continued on the next page

Ch.	Title	Main focus
10	Evaluation, Optimisation, and System Design	Evaluation methods and their limits; LLM-as-judge calibration and bias control; benchmark contamination; the performance and cost optimisation ladders ordered by gain per unit of risk; structural hallucination detection; security and responsible AI; and a repeatable framework for generative AI system design interviews.

How to Use This Book

Read the overview and diagram of a chapter to build the mental model, then work the ten interview questions before reading the answers. The questions are the point; the chapter body exists to make them answerable. Every answer is written to the depth a senior interview expects, including the trade-off the interviewer is listening for and, where relevant, the implementation.

If you are preparing on a deadline, the highest-yield path is Chapters 2, 4, and 5 — KV cache and attention variants, serving mechanics, and retrieval error attribution are where the majority of technical depth questions land. If you are interviewing for an agentic or forward-deployed role, add Chapters 6 and 9. If you are interviewing for an inference or ML systems role, Chapters 2, 4, and 8 are the core, and the roofline arithmetic in Chapter 4 is the single most reliably tested piece of reasoning in this book.

Contents

Preface i

About This Book ii

Part I Foundations 1

1 Foundations of Generative AI 2

1.1 Overview . . . 2
1.2 Discriminative versus Generative Models . . . 3
1.3 Foundation Models . . . 4
1.4 Neural Network Fundamentals for the Transformer . . . 5
1.5 Tokenization . . . 7
1.6 Embeddings . . . 9
1.7 Scaling Laws and Emergent Abilities . . . 10
1.8 The Ecosystem: Open versus Closed . . . 11
1.9 Capabilities and Limitations . . . 12
1.10 Interview Questions and Answers . . . 12

Q1.1 Explain the difference between discriminative and generative models, and describe a production system where you would deliberately use both. . . . 12
Q1.2 Your team estimated context costs using characters divided by four. After launching in Japan, users report truncated answers and your bill is 2.5x forecast. Diagnose and fix. . . . 13
Q1.3 Why does the attention formula divide by $\sqrt{d_k}$? What breaks if you remove it? . . . 15
Q1.4 Compare Pre-LN and Post-LN Transformers. Which would you pick for a 70B model and why? . . . 15
Q1.5 A stakeholder asks: "Models now have 1M-token context—can we delete our RAG pipeline?" Give your recommendation with reasoning. . . . 15
Q1.6 Explain scaling laws and Chinchilla optimality. Your company can spend a fixed compute budget—how do you allocate it, and does the answer change if the model will serve production traffic for two years? . . . 16
Q1.7 What are emergent abilities, and how would you respond to a colleague who claims they prove models will suddenly become dangerous at some scale? . . . 17
Q1.8 Your RAG system returns semantically similar but factually wrong passages. A colleague proposes a better embedding model. Is that the right fix? . . . 18
Q1.9 Walk through what happens between a user pressing Enter and the first token appearing on screen. . . . 19

Q1.10 Design the model-selection strategy for a startup with three workloads: high-volume classification, customer-facing chat, and a low-volume legal analysis tool. . . . 20

2 Transformer and LLM Architecture 23

2.1 Overview . . . 23
2.2 The Transformer Block . . . 23
2.3 Attention Variants . . . 24
2.4 Positional Encoding . . . 25
2.5 The KV Cache . . . 27
2.6 Mixture of Experts . . . 29
2.7 Efficient Attention Kernels . . . 30
2.8 Long Context . . . 31
2.9 Interview Questions and Answers . . . 32

Q2.1 Derive the KV cache size for Llama-3-70B at 8k context and explain what it means for your serving architecture. . . . 32
Q2.2 Explain MHA, MQA, GQA, and MLA. Which would you choose for a 30B model serving 5k concurrent chat users? . . . 32
Q2.3 Your team fine-tuned a model with RoPE base scaling to extend context from 8k to 64k. Benchmarks look fine but users report the model "forgets" material in long documents. Debug this. . . . 33
Q2.4 Why is FlashAttention faster if it performs the same number of FLOPs? . . . 35
Q2.5 A colleague proposes switching from a dense 70B model to a 400B-total / 40B-active MoE to "get better quality at the same cost." Evaluate this. . . . 36
Q2.6 Explain the KV cache to a backend engineer with no ML background, then explain why it causes out-of-memory errors under load. . . . 37
Q2.7 Why are LLM outputs non-deterministic even at temperature 0? Your legal client requires reproducibility—what do you tell them? . . . 38
Q2.8 Design the attention configuration for a code-completion model that must serve 50ms p99 latency with 32k context. . . . 39
Q2.9 What is the difference between the prefill and decode phases, and why does it dictate your entire serving architecture? . . . 40
Q2.10 You must serve a 7B model on a single 24GB consumer GPU with 8k context and maximum concurrency. Walk through your memory budget and optimisations. . . . 41

Part II Building Blocks 44

3 Prompt Engineering 45

3.1 Overview . . . 45
3.2 Prompt Anatomy . . . 45
3.3 System Prompt Design . . . 46
3.4 Zero-shot, Few-shot, and Example Selection . . . 48
3.5 Chain-of-Thought and Reasoning . . . 48
3.6 Structured Output . . . 49
3.7 Prompt Optimisation . . . 51
3.8 Prompt Security . . . 51

3.9 Interview Questions and Answers . . . 53
Q3.1 Your prompt works in testing and fails in production 15% of the time. Walk through your debugging process. . . . 53
Q3.2 Explain prompt injection and design a defence for a RAG system that ingests customer-uploaded PDFs. . . . 55
Q3.3 When should you use few-shot prompting versus fine-tuning? Give the decision criteria and the crossover math. . . . 57
Q3.4 A support bot leaks its system prompt when users ask cleverly. How serious is this and what do you do? . . . 58
Q3.5 Design a prompt system for a multi-tenant SaaS where each customer needs custom behaviour but you must control safety and cost. . . . 60
Q3.6 Explain chain-of-thought prompting. When does it hurt, and how would you decide whether to use it? . . . 62
Q3.7 Your extraction pipeline returns malformed JSON about 3% of the time and you have a retry loop. Critique this design. . . . 63
Q3.8 Design an A/B testing framework for prompt changes in production. . . . 65
Q3.9 How do you handle prompts that must work across multiple model providers? . . . 66
Q3.10 A prompt that worked for months suddenly degrades after a provider model update. Walk through your response. . . . 69

4 LLM Inference and Serving 72

4.1 Overview . . . 72
4.2 The Text Generation Pipeline . . . 73
4.3 Sampling Methods . . . 73
4.4 Continuous Batching . . . 75
4.5 Prefix Caching . . . 76
4.6 Speculative Decoding . . . 78
4.7 Inference Frameworks . . . 80
4.8 Serving Architecture . . . 81
4.9 Interview Questions and Answers . . . 81
Q4.1 Explain continuous batching and quantify the improvement over static batching. . . . 81
Q4.2 Your service has p50 TTFT of 200ms but p99 of 8 seconds. Diagnose. . . . 82
Q4.3 Explain PagedAttention and why it was a breakthrough. . . . 84
Q4.4 What is speculative decoding, when does it help, and when does it hurt? . . . 85
Q4.5 Design an LLM serving system for 10,000 requests/second with mixed workloads. . . . 86
Q4.6 Your inference cost is $180k/month. The CFO wants it halved without quality loss. What do you do? . . . 88
Q4.7 How do you handle streaming responses, and what breaks in production? 90
Q4.8 Compare vLLM, TensorRT-LLM, and llama.cpp. When would you choose each? . . . 92
Q4.9 Your GPU fleet OOMs during traffic spikes despite 30% headroom at steady state. Explain and fix. . . . 94
Q4.10 Explain the memory-bandwidth bound in decode and what actually helps. 95

5 Retrieval-Augmented Generation 98

5.1 Overview . . . 98

5.2 Document Parsing . . . 99
5.3 Chunking . . . 100
5.4 Retrieval . . . 101
5.5 Reranking . . . 103
5.6 Query Transformation . . . 104
5.7 Evaluation . . . 104
5.8 Interview Questions and Answers . . . 105
Q5.1 Design a RAG system for 50 million enterprise documents with per-user access control. . . . 105
Q5.2 Users say your RAG chatbot gives correct answers to first questions but nonsense on follow-ups. Diagnose. . . . 107
Q5.3 Explain the difference between bi-encoders and cross-encoders and why production systems use both. . . . 109
Q5.4 Your RAG system's answers are technically grounded but users say they "miss the point." Investigate. . . . 110
Q5.5 How would you evaluate a RAG system before and after launch? . . . 112
Q5.6 Compare chunking strategies. How would you choose for a corpus of legal contracts? . . . 113
Q5.7 Your retrieval latency is 800ms p99 and the budget is 200ms. Optimise. . . . 116
Q5.8 When would you NOT use RAG? . . . 117
Q5.9 Design a RAG evaluation and monitoring system that detects quality degradation before users complain. . . . 119
Q5.10 You must migrate a 200-million-chunk index to a new embedding model with zero downtime. Design the migration. . . . 120

Part III Advanced Systems **124**

6 AI Agents **125**

6.1 Overview . . . 125
6.2 Agent Components . . . 127
6.3 Agent Memory . . . 129
6.4 Context Management: The Real Constraint . . . 130
6.5 Multi-Agent Systems . . . 131
6.6 Human in the Loop . . . 131
6.7 Agent Frameworks . . . 132
6.8 Agent Evaluation . . . 132
6.9 Interview Questions and Answers . . . 133
Q6.1 Design an agent that resolves customer support tickets end to end. Cover architecture, tools, safety, and evaluation. . . . 133
Q6.2 Your agent works in testing but loops infinitely in production. Debug it. . . . 135
Q6.3 When should you use a multi-agent architecture versus a single agent with more tools? . . . 137
Q6.4 How do you handle tool call failures and make agents resilient? . . . 138
Q6.5 An agent with database write access is deployed. What are the security risks and how do you mitigate them? . . . 140
Q6.6 How do you evaluate an agent? Design the eval suite for a coding agent. . . . 142
Q6.7 Your agent's average cost is $0.30 per task but p99 is $12. Explain and fix. 144

Q6.8 Design the memory architecture for a personal assistant agent used daily over years. . . . 145
Q6.9 How would you migrate a rule-based workflow automation system to an agentic one? What stays rules-based? . . . 148
Q6.10 An agent in production suddenly starts failing after a model provider update. Walk through the response. . . . 149
7 Multimodal AI 153
7.1 Overview . . . 153
7.2 Vision-Language Model Architecture . . . 154
7.3 Document Intelligence and OCR . . . 156
7.4 Speech: ASR and TTS . . . 156
7.5 Image Generation . . . 157
7.6 Multimodal RAG . . . 157
7.7 Interview Questions and Answers . . . 158
Q7.1 Design a document understanding system for insurance claims: scanned forms, photos of damage, and handwritten notes. . . . 158
Q7.2 Your VLM performs well on benchmarks but fails on your customers' documents. Diagnose. . . . 160
Q7.3 Explain how vision-language models work. What is the connector and why does it matter? . . . 161
Q7.4 Design a real-time voice assistant with under 500ms response latency. . . . 162
Q7.5 Multimodal RAG over technical manuals with diagrams. Users ask questions the diagrams answer. How do you build it? . . . 165
Q7.6 Your VLM-based OCR system reports 96% accuracy but finance says the numbers are wrong. Investigate. . . . 167
Q7.7 How do you handle video understanding when a 10-minute video exceeds any context window? . . . 169
Q7.8 Compare a specialised OCR pipeline against an end-to-end VLM for invoice processing at 100,000 documents/month. . . . 171
Q7.9 Explain diffusion models and classifier-free guidance. How would you productionise image generation at scale? . . . 173
Q7.10 Design content moderation for a user-facing image generation product. . . . 176
8 Fine-Tuning and Model Adaptation 179
8.1 Overview . . . 179
8.2 Supervised Fine-Tuning . . . 180
8.3 LoRA and QLoRA . . . 181
8.4 Preference Optimisation . . . 183
8.5 Dataset Preparation . . . 185
8.6 Synthetic Data . . . 185
8.7 Evaluating a Fine-Tune . . . 186
8.8 Interview Questions and Answers . . . 186
Q8.1 When should you fine-tune versus use RAG? Give a decision framework and a case where you need both. . . . 186
Q8.2 Explain LoRA mathematically. Why does it work, and what are the practical hyperparameters? . . . 188

Q8.3 Your fine-tuned model is better at the target task but worse at everything else. Explain and fix. . . . 190
Q8.4 Design a fine-tuning pipeline for a model that must be updated monthly with new data. . . . 192
Q8.5 Compare RLHF, DPO, and KTO. Which would you choose for a product with a thumbs up/down button? . . . 195
Q8.6 You have 500 hand-labelled examples and need 10,000. How do you generate synthetic data safely? . . . 197
Q8.7 How do you serve 50 fine-tuned model variants for 50 customers cost-effectively? . . . 199
Q8.8 Your fine-tune shows 15% improvement on your eval set but users report no difference. What went wrong? . . . 200
Q8.9 Your SFT run's training loss is decreasing smoothly but eval quality is flat or worse. Debug it. . . . 202
Q8.10 How would you distil a frontier model into a 3B model you can serve yourself? Cover data, method, legality, and evaluation. . . . 203

Part IV Production **207**

9 Production Generative AI Systems **208**

9.1 Overview . . . 208
9.2 API Design . . . 209
9.3 Guardrails . . . 210
9.4 Caching . . . 211
9.5 Rate Limiting and Cost Control . . . 211
9.6 Observability . . . 212
9.7 Deployment . . . 213
9.8 Interview Questions and Answers . . . 213
Q9.1 Design the complete production architecture for an enterprise AI assistant serving 50,000 employees. . . . 213
Q9.2 Your LLM costs jumped 300% overnight with no traffic increase. Investigate. 214
Q9.3 Design a guardrail system. What do you check, where, and how do you handle guardrail failures? . . . 216
Q9.4 How do you monitor LLM quality in production when there is no ground truth? . . . 219
Q9.5 Design a semantic caching system. What are the risks and how do you mitigate them? . . . 220
Q9.6 A provider outage takes down your main model. Design the failover strategy. 223
Q9.7 How do you handle PII in an LLM pipeline that must comply with GDPR? 225
Q9.8 Design the deployment and autoscaling strategy for self-hosted LLM inference on Kubernetes. . . . 227
Q9.9 A customer reports the model "stops mid-sentence" intermittently. Walk through the investigation. . . . 229
Q9.10 Design the on-call runbook for a generative AI service. What alerts fire, and what does the responder do? . . . 230

10 Evaluation, Optimisation, and System Design **234**

10.1 Overview . . . 234

10.2 Evaluation Methods . . . 235
10.3 Benchmarks . . . 237
10.4 Performance Optimisation . . . 237
10.5 Cost Optimisation . . . 238
10.6 Hallucination Detection . . . 239
10.7 Security and Responsible AI . . . 239
10.8 System Design Interview Framework . . . 239
10.9 Interview Questions and Answers . . . 240
Q10.1 Design an end-to-end evaluation system for a customer-facing LLM product from scratch. . . . 240
Q10.2 Design a production RAG system for a legal firm: 10 million documents, strict accuracy requirements, full auditability. . . . 243
Q10.3 How do you detect and reduce hallucinations in a production RAG system? 245
Q10.4 Your CFO wants to cut AI spend 50% without hurting quality. Present your plan. . . . 247
Q10.5 Design an AI system that summarises 10,000 documents daily with 99.9% reliability. . . . 248
Q10.6 Compare LLM-as-judge with human evaluation. When is each appropriate, and how do you validate a judge? . . . 251
Q10.7 Walk me through designing an AI coding assistant end to end. . . . 253
Q10.8 What are the most important lessons you would give someone building their first production generative AI system? . . . 255
Q10.9 You are launching a new product with no production traffic. How do you build an evaluation set from a cold start? . . . 256
Q10.10 Design the process for deciding whether to deploy a quantized model. FP16, FP8, INT8, or INT4? . . . 257
Closing Notes **261**

PART I

Foundations

Before a model can answer a question, the question has to become tokens and the tokens have to become computation. This part covers that conversion and its price: what generative modelling actually estimates, why the Transformer displaced everything before it, what a token costs in each language and each modality, and how every architectural choice in a modern LLM — rotary positions, grouped-query attention, sparse experts, fused attention kernels — is an answer to a memory or bandwidth problem rather than an intelligence one.

CHAPTER 1

Foundations of Generative AI

Subscribe → aiengineeringinsider.substack.com/subscribe

What a generative model actually estimates, why the Transformer won, and the three numbers—tokens, parameters, and context—that price every system you will ever design on top of one.

1.1 Overview

Almost every production incident in a generative AI system traces back to a misunderstanding at this level. A team blames the model for truncating an answer when the real cause is that their tokenizer expanded a Japanese document to three times the token count they budgeted. Another team pays for a 70B model when the task is extraction and an 8B model with a good prompt would have matched it. A third ships a retrieval system whose embedding model was never trained on their domain vocabulary and wonders why recall is poor.

This chapter builds the mental model that prevents those failures. We define what generative modelling *is* in probabilistic terms, walk the Transformer components that make it computationally tractable, and then work through the four quantities that dominate every downstream design decision: how text becomes tokens, how tokens become vectors, how model capability scales with compute, and where the whole approach breaks down.

Core Definition

A **generative model** learns to approximate the joint probability distribution $p(x)$ over a data space, so that it can *sample* new points from that space. A **discriminative model** learns only the conditional $p(y \mid x)$—a decision boundary. Autoregressive language models factor the joint distribution using the chain rule of probability:

$$p(x_1, x_2, \ldots, x_T) \;=\; \prod_{t=1}^{T} p(x_t \mid x_{<t})\,,$$

which reduces "model all of language" to "predict the next token, repeatedly." Every capability an LLM exhibits—reasoning, translation, code synthesis—is an emergent consequence of optimising that one objective at scale.

⋆ Key Insight

The chain-rule factorisation is why LLMs are so general and why they hallucinate. The model is never trained to be *correct*; it is trained to be *likely*. A fluent falsehood and a fluent truth receive identical treatment from the loss function unless the training data disambiguates them. Every grounding technique in this book—RAG, tool use, verification—exists to inject

a correctness signal that pretraining does not provide.

1.1.1 The Generative AI Stack

Before the details, it helps to see where each chapter of this book sits. Figure 1.1 shows the layered architecture that every serious generative AI product converges on, regardless of vendor.

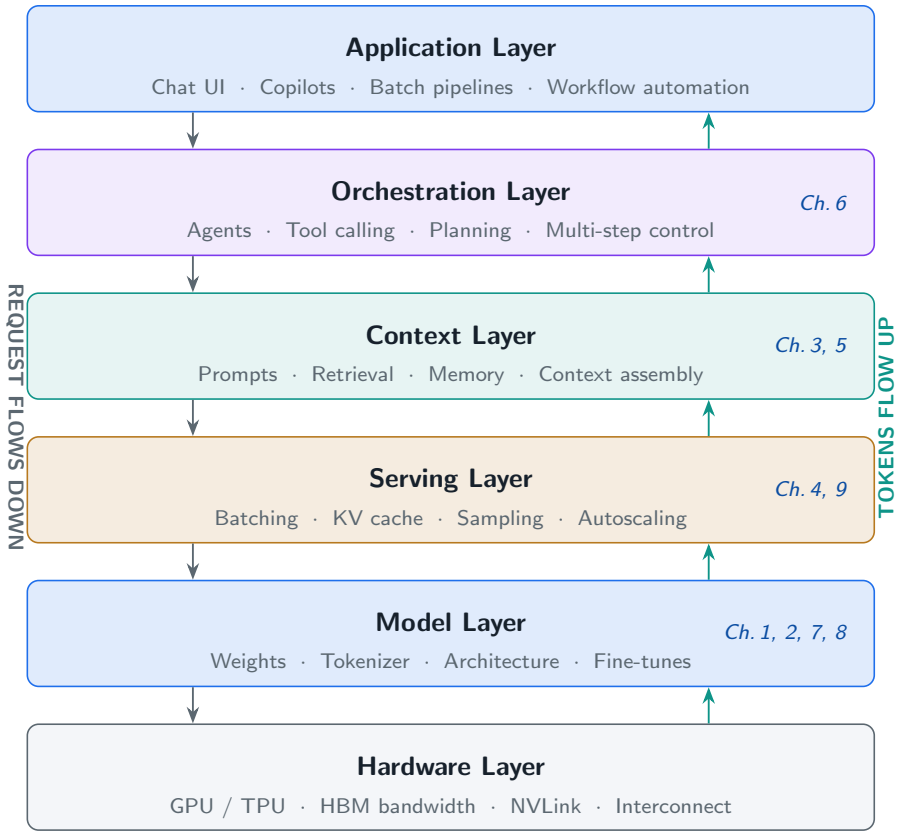


Figure 1.1: The generative AI stack. Each layer consumes the guarantees of the layer beneath it and exposes a narrower, safer contract upward. Chapter numbers on the right mark where each layer is covered in depth.

1.2 Discriminative versus Generative Models

The distinction is not academic—it changes what you can build, how you evaluate, and what failure looks like.

A discriminative classifier trained to label support tickets as `billing` or `technical` learns a boundary in feature space. It cannot write a ticket, cannot explain its decision beyond feature attribution, and degrades predictably when the input distribution shifts. Its failure mode is *misclassification*, and it is measurable with a confusion matrix.

A generative model trained on the same corpus learns the shape of the entire distribution. It can classify (by conditioning), summarise, rewrite, translate, and extract—all without task-specific heads. Its failure mode is *plausible fabrication*, which no confusion matrix detects.

Dimension	Discriminative	Generative
Learns	$p(y \mid x)$ — a decision boundary	$p(x)$ or $p(x \mid c)$ — the data distribution
Typical models	Logistic regression, XGBoost, BERT classifiers, CNNs	GPT-family, Llama, Claude, diffusion models, VAEs
Data hunger	Modest; thousands of labelled examples	Extreme; trillions of unlabelled tokens
Task transfer	Retrain per task	Prompt or lightly fine-tune per task
Failure mode	Misclassification (detectable)	Hallucination (fluent, undetectable without grounding)
Evaluation	Accuracy, F1, AUC	Perplexity, human preference, task-specific rubrics, LLM-as-judge
Inference cost	Microseconds, CPU-friendly	Milliseconds to seconds per token, GPU-bound
Use when	Labels are cheap, latency is tight, task is fixed	Task is open-ended, labels are scarce, output is text

From the Field

The most common architecture mistake I see in 2026 is using a 70B parameter generative model to do a job a 400M parameter encoder does better, faster, and for 1/200th the cost. Intent classification, PII detection, toxicity screening, and semantic routing are all discriminative problems. Use a fine-tuned ModernBERT or a small cross-encoder. Save the generative budget for the parts of the pipeline that actually need to *write* something.

1.3 Foundation Models

A **foundation model** is a model trained on broad data at scale, using self-supervision, that is designed to be adapted rather than used directly (Bommasani et al., 2021). Three properties define the category.

Scale Trained on corpora measured in trillions of tokens with compute budgets measured in exaFLOP-days. The scale is what produces generality.

Self-supervision The training signal comes from the data itself—next-token prediction, masked-token reconstruction, or noise removal—so no human labelling bottleneck constrains corpus size.

Adaptability A single set of weights serves hundreds of downstream tasks via prompting, retrieval augmentation, adapters, or full fine-tuning. This is the economic argument: amortise one enormous training run across every application.

Foundation models come in three architectural families, and the interview question "which would you choose and why?" is answered by matching the family to the information flow the task requires.

Family	Objective	Best at	Examples
Decoder-only	Next-token prediction, causal mask	Open-ended generation, chat, code, agentic control	GPT-4o, Claude, Llama 3.x, Qwen, DeepSeek-V3
Encoder-only	Masked-token prediction, bidirectional	Classification, embeddings, reranking, extraction	BERT, RoBERTa, ModernBERT, DeBERTa
Encoder–decoder	Denoising / span corruption, seq2seq	Translation, summarisation with a fixed source	T5, FLAN-T5, BART, mT5
Diffusion	Iterative denoising of latents	Images, audio, video; high-fidelity continuous data	Stable Diffusion, Flux, SDXL

1.4 Neural Network Fundamentals for the Transformer

You do not need to derive backpropagation in an interview, but you do need to explain *why* each Transformer component exists. Each one solves a specific failure of the architecture that preceded it.

1.4.1 Self-Attention: The Core Intuition

Recurrent networks processed tokens sequentially, which meant that information from token 1 reached token 500 only by surviving 499 lossy state updates. Long-range dependencies decayed. Training could not be parallelised across the sequence.

Self-attention replaces sequential state passing with direct, learned, all-pairs communication. Every token emits three vectors:

Query (Q) "What information am I looking for?"

Key (K) "What information do I offer?"

Value (V) "Here is the content I contribute if you attend to me."

The output for each position is a weighted average of all value vectors, where the weights come from query–key similarity:

$$\text{Attention}(Q, K, V) \;=\; \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}} + M\right) V.$$

Two details carry interview weight. The $\sqrt{d_k}$ divisor prevents the dot products from growing with dimension and saturating the softmax into a near-one-hot distribution with vanishing gradients (Vaswani et al., 2017). The mask M sets future positions to $-\infty$ in decoder-only models, enforcing causality so that position t cannot see $t+1$—which is exactly what makes the KV cache of Chapter 2 possible.

! Pitfall / Warning

Self-attention is $O(T^2 d)$ in time and, if materialised naively, $O(T^2)$ in memory. At $T = 128{,}000$ the attention matrix alone would be roughly 16 billion entries per head per layer. This is why FlashAttention (which never materialises the matrix) is not an optimisation you

may skip—it is a precondition for long context. See Section 2.7.

1.4.2 Multi-Head Attention

A single attention operation computes one weighted average, which forces the model to blend all relational signals into one distribution. Multi-head attention splits d_{model} into h subspaces, runs attention independently in each, and concatenates. Empirically, different heads specialise: some track syntactic dependencies, some track coreference, some perform the induction (copy-the-pattern) behaviour that underlies in-context learning.

Scaled dot-product attention -- the reference implementation

```
import torch
import torch.nn.functional as F

def multi_head_attention(x, w_qkv, w_out, n_heads, causal=True):
    """x: (batch, seq, d_model). Returns (batch, seq, d_model)."""
    B, T, D = x.shape
    head_dim = D // n_heads

    # One fused projection is cheaper than three separate matmuls.
    qkv = x @ w_qkv                          # (B, T, 3D)
    q, k, v = qkv.chunk(3, dim=-1)

    # (B, n_heads, T, head_dim) -- heads become a batch dimension.
    reshape = lambda t: t.view(B, T, n_heads, head_dim).transpose(1, 2)
    q, k, v = reshape(q), reshape(k), reshape(v)

    scores = (q @ k.transpose(-2, -1)) / (head_dim ** 0.5)   # (B, H, T, T)

    if causal:
        mask = torch.triu(torch.ones(T, T, device=x.device), diagonal=1).bool()
        scores = scores.masked_fill(mask, float("-inf"))

    out = F.softmax(scores, dim=-1) @ v                      # (B, H, T, head_dim)
    out = out.transpose(1, 2).reshape(B, T, D)
    return out @ w_out
```

1.4.3 Feed-Forward Networks

Attention moves information *between* positions; it performs no nonlinear computation *within* a position. The position-wise feed-forward network supplies that:

$$\text{FFN}(x) = W_2\,\sigma(W_1 x + b_1) + b_2,$$

typically with an inner dimension of $4d_{\text{model}}$ and a gated activation (SwiGLU in most modern models). This block holds roughly two-thirds of a dense Transformer's parameters, which is why Mixture-of-Experts targets it for sparse activation (Section 2.6).

1.4.4 Residual Connections and Layer Normalisation

Residual connections ($x + f(x)$) give gradients an identity path to every layer, which is what makes 80- and 120-layer stacks trainable at all (He et al., 2016). Layer normalisation stabilises activation statistics per token.

The placement matters and is a favourite interview probe. **Post-LN** (the original 2017 design) normalises after the residual add and requires learning-rate warmup to avoid divergence. **Pre-LN** normalises the branch input, keeps the residual stream clean, and trains stably without careful warmup—so every modern large model uses it. RMSNorm (Zhang & Sennrich, 2019) drops the mean-centring term, saving compute for no measurable quality loss, and is now the default in Llama, Qwen, Mistral, and DeepSeek.

1.5 Tokenization

Tokenization is the single most under-appreciated source of production bugs in generative AI. It determines your cost, your effective context length, your multilingual fairness, and a surprising number of your model's reasoning failures.

Tokenization

The reversible mapping from a text string to a sequence of integer IDs drawn from a fixed vocabulary. Subword algorithms sit between two bad extremes: character tokens produce sequences too long to attend over, and word tokens produce vocabularies too large to embed and cannot represent unseen words.

1.5.1 The Three Algorithms

Byte-Pair Encoding (BPE) starts with individual bytes and greedily merges the most frequent adjacent pair, repeating until the vocabulary target is reached (Sennrich et al., 2016). **Byte-level BPE**—used by GPT-family models and Llama—operates on raw UTF-8 bytes, which guarantees that no input is ever out-of-vocabulary, including emoji, Cyrillic, and corrupted encodings.

WordPiece (BERT) merges the pair that maximises corpus likelihood rather than raw frequency, a subtly different criterion that tends to produce more linguistically coherent subwords.

SentencePiece is not a merge algorithm but a *framework* that treats the input as a raw Unicode stream—spaces included, encoded as `U+2581`—so it needs no language-specific pre-tokenizer. It can run either BPE or the Unigram algorithm underneath. This is what makes it the standard for multilingual and non-space-delimited languages such as Japanese and Thai.

Algorithm	Merge criterion	Properties	Used by
BPE	Highest pair frequency	Fast, deterministic, simple; can split words oddly at boundaries	GPT-2, RoBERTa
Byte-level BPE	Frequency over UTF-8 bytes	Zero OOV by construction; inflates token count for non-Latin scripts	GPT-4o, Llama 3, Mistral
WordPiece	Max corpus likelihood gain	Cleaner morphological splits; `##` continuation marker	BERT, DistilBERT, ELECTRA
Unigram (SentencePiece)	Prune tokens by likelihood loss	Probabilistic, supports subword regularisation sampling	T5, ALBERT, XLNet

1.5.2 Token Counting and Its Consequences

For English prose the working heuristic is 1 token ≈ 4 characters ≈ 0.75 words. That heuristic is dangerously wrong for everything else.

Content type	Tokens per 1k chars	Engineering consequence
English prose	≈ 250	Baseline for all capacity planning
Source code	≈ 320–400	Indentation and punctuation tokenize poorly; budget 1.5×
JSON / XML payloads	≈ 400–500	Structural characters dominate; prefer compact schemas
CJK (Chinese, Japanese)	≈ 600–900	Same document costs 2–3× more than its English translation
Base64 / hashes / UUIDs	≈ 700–1000	Near-worst case; never put raw blobs in a prompt

Auditing real token cost before you commit to a context budget

```
import tiktoken

enc = tiktoken.get_encoding("o200k_base")    # GPT-4o family

def audit(label: str, text: str, price_per_mtok: float = 2.50) -> dict:
    ids = enc.encode(text)
    n = len(ids)
    return {
        "label": label,
        "chars": len(text),
        "tokens": n,
        "chars_per_token": round(len(text) / max(n, 1), 2),
        "usd_per_1k_calls": round(n / 1_000_000 * price_per_mtok * 1000, 4),
    }

samples = {
    "english":  "The quick brown fox jumps over the lazy dog. " * 20,
    # "He learns a new technology every day." in Japanese, written with
    # \u escapes so this listing stays ASCII. In practice, load real locale
    # samples from a fixtures file rather than embedding them in source.
```

```
    "japanese": ("\u5f7c\u306f\u6bce\u65e5\u65b0\u3057\u3044"
                 "\u6280\u8853\u3092\u5b66\u3093\u3067\u3044\u307e\u3059\u3002") * 20,
    "json":     '{"user_id": "a3f9-22b1", "status": "active"}, ' * 20,
}

for label, text in samples.items():
    print(audit(label, text))

# Typical result: Japanese costs ~2.4x more tokens than English for the
# same *information*, and JSON costs ~1.7x more than the prose equivalent.
# Both facts belong in your capacity model, not in a post-mortem.
```

! Pitfall / Warning

Never estimate tokens with `len(text.split())` or a character divisor in production. Count with the actual tokenizer for the actual model. A pricing model built on `chars / 4` under-reports Japanese traffic by more than half; the first month with real international users turns that into a budget overrun and a wave of context-overflow errors.

1.5.3 Context Window

The **context window** is the maximum number of tokens the model can attend over in a single forward pass—prompt plus generated output together. It is a hard architectural limit, not a soft preference.

Three practical consequences that interviewers probe:

1. **Advertised is not usable.** A 128k window must also hold your system prompt, few-shot examples, retrieved chunks, tool schemas, conversation history, *and* the reserved output budget. Plan for roughly 70–80% usable.
2. **Quality degrades before capacity does.** Models retrieve information placed at the beginning and end of a long context far more reliably than information in the middle—the "lost in the middle" effect (Liu et al., 2024). Put the critical instruction last.
3. **Cost is superlinear in prefill.** Attention is quadratic in sequence length, so doubling the prompt more than doubles time-to-first-token. Long context is not a substitute for retrieval; it is a more expensive alternative to it.

1.6 Embeddings

An **embedding** is a dense vector that places an input in a geometric space where distance encodes semantic relatedness. Two distinct kinds appear in a production system and conflating them is a classic interview trap.

Token embeddings are the model's internal lookup table—one learnable vector per vocabulary entry, retrieved at the input layer and refined by every subsequent Transformer block. They are contextual only after the first layer.

Sentence or document embeddings are the output of a dedicated model (usually an encoder trained with a contrastive objective) that maps a whole passage to a single vector for similarity search. These are what your vector database stores.

Embedding type	Produced by	Used for
Token embedding	Model input layer	Internal representation; never queried directly
Semantic (text) embedding	Contrastive encoder	Vector search, clustering, deduplication, classification
Cross-modal embedding	CLIP / SigLIP-style dual encoder	Text-to-image retrieval, zero-shot classification
Late-interaction (multi-vector)	ColBERT / ColPali	High-recall retrieval; one vector per token, higher index cost

⋆ **Key Insight**

Cosine similarity measures direction, not truth. Two passages that both discuss "insurance claim denial" will sit close together whether one *approves* the claim and the other *denies* it. Dense retrieval is a recall mechanism, not a precision mechanism—which is precisely why the reranking stage of Section 5.5 is non-optional in production RAG.

1.7 Scaling Laws and Emergent Abilities

Kaplan et al. (2020) showed that test loss falls as a smooth power law in parameters N, dataset size D, and compute C—smooth enough that a lab can predict a large model's loss from small-scale runs before spending the training budget.

Hoffmann et al. (2022), the "Chinchilla" work, corrected the optimal allocation: for a fixed compute budget, parameters and training tokens should scale roughly *equally*, implying about 20 tokens per parameter. Most models before that paper were badly under-trained.

From the Field

Chinchilla optimality is a *training-compute* criterion, and it is the wrong objective if you are going to serve the model. A model trained far past Chinchilla—Llama 3 8B saw roughly 15 trillion tokens, about 1875 tokens per parameter—costs more to train but is dramatically cheaper to *serve* at its quality level. When the model runs for a year in production, inference cost dominates the total bill. This is why the industry moved to deliberately "over-training" small models.

Emergent abilities are capabilities absent in smaller models that appear, seemingly discontinuously, past a scale threshold: multi-step arithmetic, instruction following, chain-of-thought reasoning, in-context learning from a handful of examples. Wei et al. (2022) documented the phenomenon; Schaeffer et al. (2023) argued convincingly that much of the apparent discontinuity is an artefact of using all-or-nothing metrics like exact-match accuracy, and that continuous metrics reveal smooth improvement. Being able to state both sides of this is a strong signal in a senior interview.

1.8 The Ecosystem: Open versus Closed

Provider	Access model	Characteristic strengths and design notes
OpenAI	Closed API	Broad tool/function-calling ecosystem, structured outputs, strong multimodal and reasoning tiers
Anthropic	Closed API	Long-context reliability, constitutional-AI safety training, strong agentic tool use and computer use
Google	Closed API	Very long context, native multimodality across text/image/audio/video, TPU-backed economics
Meta (Llama)	Open weights	The de facto self-hosting baseline; the largest fine-tuning and tooling ecosystem
DeepSeek	Open weights	MoE efficiency (MLA + fine-grained experts), reasoning distillation, aggressive cost per token
Qwen	Open weights	Strong multilingual coverage, dense and MoE variants across many sizes, permissive licensing
Mistral	Mixed	Efficient dense and sparse models, European data-residency positioning

! Pitfall / Warning

Model names, prices, context limits, and benchmark scores change monthly. Treat every specific figure in this book as a snapshot for reasoning about *trade-offs*, and verify current numbers against the provider's own documentation before you put them in a design document.

1.8.1 The Build-versus-Buy Decision

Factor	Closed API	Open weights, self-hosted
Time to first demo	Hours	Days to weeks
Unit economics	Linear in tokens; cheap at low volume	Fixed GPU cost; cheaper past a crossover point
Data residency	Vendor-controlled; contractual guarantees	Full control; required for many regulated workloads
Customisation	Prompting, limited fine-tuning	Full fine-tuning, LoRA, quantization, custom kernels
Latency floor	Network round trip plus queueing	Controllable; co-locate with your data
Version stability	Vendor may deprecate or silently update	You pin the checkpoint forever
Ops burden	Near zero	GPU capacity, autoscaling, upgrades, on-call

✓ Best Practice

Start on a closed API to validate that the product works at all. Instrument token volume from day one. Compute the crossover point honestly—including the engineer-months of serving work, not just GPU rental—and migrate the highest-volume, lowest-complexity route

first. Hybrid routing (Section 10.5) usually beats an all-or-nothing decision.

1.9 Capabilities and Limitations

An honest account of the limits is what separates a senior candidate from a enthusiastic one.

Hallucination The model optimises likelihood, not truth. Mitigate with retrieval grounding, citation requirements, and verification passes—never with "please be accurate" in the prompt.

Knowledge cutoff Parametric knowledge is frozen at training time. Anything time-sensitive must arrive through retrieval or tools.

No native arithmetic Numbers are tokens. Multi-digit arithmetic is pattern matching over token sequences and fails unpredictably. Route calculation to a code interpreter.

Context degradation Effective attention over very long contexts is worse than nominal capacity implies; position matters.

Non-determinism Even at temperature 0, floating-point non-associativity under varying batch composition makes outputs non-bitwise-reproducible on GPU.

Prompt injection There is no reliable in-band separation between instructions and data. Any text the model reads is a potential instruction (Section 3.8).

Cost and latency Generation is memory-bandwidth-bound and sequential. You cannot parallelise your way out of a long output.

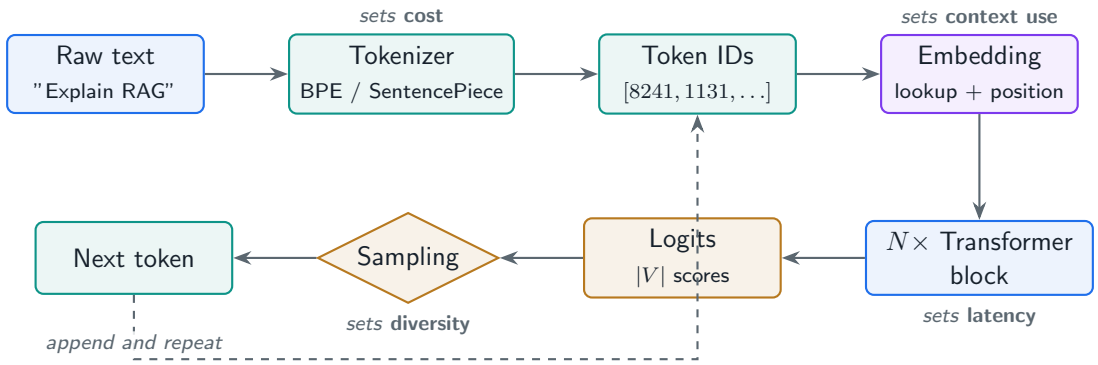


Figure 1.2: From raw text to sampled token—the full forward path, with the four quantities that dominate cost annotated at the stage where they are set.

1.10 Interview Questions and Answers

Q1.1 Explain the difference between discriminative and generative models, and describe a production system where you would deliberately use both.

The distinction. A discriminative model estimates $p(y \mid x)$: it learns a boundary that separates classes and cannot produce new samples. A generative model estimates $p(x)$ or $p(x \mid c)$: it learns the data distribution well enough to sample from it. Autoregressive LLMs are generative models that factor the joint distribution via the chain rule, $p(x_{1:T}) = \prod_t p(x_t \mid$

$x_{<t}$).

The production system. Consider a customer support automation platform handling 2 million tickets a month. A pure-LLM design—send every ticket to a frontier model—is both slow and expensive, and it applies a 10^{11}-parameter generative model to problems that a 10^{8}-parameter classifier solves better. The correct architecture is a cascade:

1. **Discriminative router** (fine-tuned ModernBERT, ≈150M params, CPU-servable, p99 under 15 ms): classify intent into one of 40 categories and predict a complexity score. About 55% of tickets are routine and resolvable by a deterministic template.
2. **Discriminative safety gate** (small cross-encoder): PII detection and toxicity screening. This must be a separate model—you cannot ask the generative model to police its own input reliably.
3. **Generative tier** for the remaining 45%: an LLM with RAG over the knowledge base drafts the response.
4. **Discriminative verifier**: an NLI cross-encoder scores whether the draft is entailed by the retrieved evidence. Low-entailment drafts escalate to a human.

Why this is the right answer. The two model families have complementary failure modes. Discriminative models fail by misclassification, which is detectable with a labelled holdout and a confusion matrix. Generative models fail by fluent fabrication, which is invisible to accuracy metrics. Wrapping the generative core in discriminative gates on both sides gives you measurable control over a component that is otherwise unmeasurable. In practice this cascade cuts cost per ticket by 60–80% versus routing everything to a frontier model, and—more importantly—it gives you a dial (the entailment threshold) to trade automation rate against error rate.

Q1.2 Your team estimated context costs using characters divided by four. After launching in Japan, users report truncated answers and your bill is 2.5x forecast. Diagnose and fix.

Diagnosis. The `chars / 4` heuristic is calibrated on English text tokenized by a byte-level BPE vocabulary whose merges were learned from a predominantly English corpus. Japanese text has two compounding problems. First, each character carries far more information, so a Japanese document has perhaps a third the character count of its English translation—which makes the heuristic under-count before tokenization even runs. Second, and much worse, few Japanese multi-character sequences earned merges in the vocabulary, so Japanese text tokenizes at close to the byte level. A single kanji is three UTF-8 bytes and may consume two or three tokens on its own.

The net effect: for the same semantic content, Japanese consumes roughly 2–3× the tokens of English. Your prompt assembly stage silently overflowed the context window, retrieval chunks were dropped from the tail, and the output budget was squeezed—producing truncation. Your cost forecast, built on the same heuristic, missed by the same factor.

The fix, in order of urgency.

1. **Stop estimating.** Replace every heuristic with the real tokenizer (`tiktoken` for OpenAI models, the HuggingFace `AutoTokenizer` for open models) at the point where the context is assembled.

2. **Make the budget a first-class object.** Build the prompt against an explicit token budget with a declared priority order, and count as you go.
3. **Fail loudly.** If the budget cannot be met, raise—do not silently truncate. Silent truncation is what turned a capacity bug into a quality bug that took a week to find.
4. **Re-forecast per locale.** Cost models must be segmented by language. Measure tokens-per-character on a sample of real traffic per locale.

A context budget that cannot silently truncate

```
from dataclasses import dataclass, field

@dataclass
class ContextBudget:
    limit: int                      # model context window
    reserved_output: int            # tokens you must leave for generation
    encode: callable                # the REAL tokenizer's encode function
    used: int = 0
    parts: list = field(default_factory=list)

    @property
    def remaining(self) -> int:
        return self.limit - self.reserved_output - self.used

    def add(self, name: str, text: str, required: bool = False) -> bool:
        n = len(self.encode(text))
        if n > self.remaining:
            if required:
                raise ValueError(
                    f"Required part {name!r} needs {n} tokens, "
                    f"only {self.remaining} remain. Reduce retrieval k "
                    f"or switch to a larger-context model."
                )
            return False            # optional part dropped, caller can log it
        self.used += n
        self.parts.append((name, n, text))
        return True

# Priority order is explicit: system prompt and question are non-negotiable,
# retrieved chunks are best-effort and fill whatever is left.
budget = ContextBudget(limit=128_000, reserved_output=4_000, encode=enc.encode)
budget.add("system", SYSTEM_PROMPT, required=True)
budget.add("question", user_question, required=True)

dropped = 0
for chunk in retrieved_chunks:              # already reranked, best first
    if not budget.add(f"chunk:{chunk.id}", chunk.text):
        dropped += 1
metrics.gauge("rag.chunks_dropped", dropped)   # alert if this is ever nonzero
```

The systemic lesson. Any quantity that determines both correctness and cost deserves a measurement, not an estimate. The `chars / 4` rule is a mental shortcut for a whiteboard,

and it should never survive contact with a pricing spreadsheet.

Q1.3 Why does the attention formula divide by $\sqrt{d_k}$? What breaks if you remove it?

The mechanism. Assume the query and key components are independent with mean 0 and variance 1. Their dot product $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$ is a sum of d_k independent zero-mean terms, so its variance is d_k and its standard deviation is $\sqrt{d_k}$. With $d_k = 128$, raw scores routinely span ± 30 or more.

What breaks. Softmax over scores with that spread saturates: one entry receives essentially all the probability mass and the rest receive near-zero. The gradient of softmax at an output p_i is proportional to $p_i(1 - p_i)$, which vanishes as $p_i \to 1$ or $p_i \to 0$. So the attention layer stops learning—and it does so worse at higher dimension, which is exactly backwards from what you want.

Dividing by $\sqrt{d_k}$ normalises the score variance back to approximately 1 regardless of head dimension, keeping the softmax in its responsive regime and making the architecture scale-invariant with respect to d_k.

Follow-up you should anticipate. "Is $\sqrt{d_k}$ always right?" Not quite. It assumes unit-variance inputs, which holds early in training but drifts. Some architectures learn the temperature instead, and QK-normalisation (applying RMSNorm to Q and K before the dot product) has become common in very large models precisely because it enforces the variance assumption rather than assuming it, improving training stability at scale.

Q1.4 Compare Pre-LN and Post-LN Transformers. Which would you pick for a 70B model and why?

The two placements.

- **Post-LN** (Vaswani et al., 2017): $x_{\ell+1} = \mathrm{LN}(x_\ell + f(x_\ell))$. Normalisation sits *on* the residual stream.
- **Pre-LN**: $x_{\ell+1} = x_\ell + f(\mathrm{LN}(x_\ell))$. Normalisation sits on the *branch*; the residual stream is untouched.

Why it matters at depth. In Post-LN, the identity path passes through a LayerNorm at every layer, so the gradient is repeatedly rescaled. Xiong et al. (2020) showed the expected gradient norm at initialisation grows with depth, producing instability that requires a long learning-rate warmup to survive. In Pre-LN the residual stream is a clean additive path from the output back to layer 1, so gradient magnitude is roughly depth-independent and training is stable without careful warmup schedules.

The choice for 70B. Pre-LN, with RMSNorm rather than LayerNorm. At 80 layers, Post-LN would demand extensive warmup tuning and would still risk loss spikes—and a loss spike at that scale costs six figures in wasted compute. Every production model in this size class (Llama, Qwen, Mistral, DeepSeek) uses Pre-LN + RMSNorm for exactly this reason.

The honest trade-off. Pre-LN's residual stream grows in variance with depth, which can make later layers contribute proportionally less—a mild representation-collapse effect. The standard mitigations are a final normalisation before the output head (universal in practice) and, in some models, scaled residual initialisation. Naming this trade-off, rather than presenting Pre-LN as strictly superior, is what distinguishes a senior answer.

Q1.5 A stakeholder asks: "Models now have 1M-token context—can we delete our RAG pipeline?" Give your recommendation with reasoning.

Recommendation: no, and the reasons are economic before they are technical.
Cost. Prefill compute scales quadratically in sequence length for the attention component. Stuffing a 500k-token corpus into every request means paying to process the entire corpus on every question. RAG pays an embedding cost once at ingestion and a retrieval cost of a few milliseconds per query, then sends 4k tokens. At a million queries a month the difference is not a rounding error—it is two orders of magnitude on the inference bill.
Latency. Time-to-first-token is dominated by prefill. A 500k-token prefill is tens of seconds on any hardware you can rent. No interactive product survives that.
Quality. Long-context recall is uneven. Information placed mid-context is retrieved substantially less reliably than information at the edges (Liu et al., 2024), and distractor passages measurably degrade answer accuracy. Retrieval is not merely a compression step—it is a *precision* step that removes distractors before the model sees them.
Operations. A vector index supports incremental update, per-document access control, deletion for GDPR compliance, and citation back to a source. A 1M-token prompt supports none of those. "Delete this customer's data" is a one-line operation against an index and an impossible one against a prompt.
Where long context genuinely wins. It is the right tool when the working set is a single coherent artefact that must be reasoned over holistically: a 900-page contract where clause 3 modifies clause 847, a full codebase for a refactoring task, a long meeting transcript. Chunking destroys the cross-references those tasks depend on.
The architecture I would actually propose. Keep RAG as the default path. Add a *long-context escalation tier*: when the reranker's top-k scores are uniformly low—meaning retrieval could not find a confident answer—or when the query is classified as document-holistic, escalate to a long-context call over the full source. Measure the escalation rate. In most deployments it lands between 3% and 8%, and you have captured the quality benefit for 5% of the cost.

Q1.6 Explain scaling laws and Chinchilla optimality. Your company can spend a fixed compute budget—how do you allocate it, and does the answer change if the model will serve production traffic for two years?

Scaling laws. Kaplan et al. (2020) established that cross-entropy loss falls as a power law in parameters N, tokens D, and compute C, with remarkably smooth and extrapolable curves. This is what lets a lab predict a frontier model's loss from a ladder of small runs.
Chinchilla. Hoffmann et al. (2022) held compute fixed and varied the N/D split, finding the earlier work had systematically under-weighted data. The compute-optimal ratio is roughly **20 training tokens per parameter**. Under that criterion, a 6×10^{23} FLOP budget is best spent on approximately a 67B model trained on 1.4T tokens—not on a 175B model trained on 300B tokens, which was the prevailing practice.
The allocation, if training loss is the only objective. Apply Chinchilla: $N \approx \sqrt{C/(6 \times 20)}$, $D \approx 20N$. Reserve 10–15% of the budget for ablations and restarts, because a divergence at scale that you cannot recover from is the most expensive outcome available.
Why two years of serving changes everything. Chinchilla optimises *training* compute. It says nothing about inference. If the model serves production traffic, total cost of ownership

is

$$C_{\text{total}} = C_{\text{train}} + C_{\text{inference}} \times \text{lifetime requests},$$

and for any successful product the second term dominates within months. Inference cost scales with N (parameters loaded per token) and is entirely independent of D (tokens seen during training, a one-time cost).

So the correct move is to train a *smaller* model on *more* tokens than Chinchilla prescribes—deliberately past the compute-optimal point. Llama 3 8B was trained on roughly 15T tokens, about 1875 tokens per parameter, nearly 100× the Chinchilla ratio. That is wasteful by the training-compute criterion and clearly correct by the TCO criterion.

Total cost of ownership: where over-training pays for itself

```python
def tco(params_b, train_tokens_t, monthly_requests, months,
        tokens_per_req=500, gpu_hour_usd=2.0):
    """Rough TCO comparison. Training FLOPs ~ 6*N*D; inference ~ 2*N per
    ↪ token."""
    N = params_b * 1e9
    D = train_tokens_t * 1e12

    train_flops = 6 * N * D
    # ~4e14 effective FLOP/s per GPU at realistic MFU
    train_usd = train_flops / 4e14 / 3600 * gpu_hour_usd

    infer_flops = 2 * N * tokens_per_req * monthly_requests * months
    infer_usd = infer_flops / 4e14 / 3600 * gpu_hour_usd

    return {"train_musd": round(train_usd / 1e6, 2),
            "infer_musd": round(infer_usd / 1e6, 2),
            "total_musd": round((train_usd + infer_usd) / 1e6, 2)}

# Chinchilla-optimal 70B (1.4T tokens) vs over-trained 8B (15T tokens),
# both serving 50M requests/month for 24 months:
print("70B chinchilla:", tco(70, 1.4, 50e6, 24))
print("8B  overtrained:", tco(8, 15.0, 50e6, 24))
# The 8B costs more per unit of training quality and far less in total,
# because inference is ~9x cheaper and runs 50M times a month for two years.
```

The one-sentence version for an interviewer: Chinchilla tells you how to spend a training budget; TCO tells you how to spend a business budget, and they disagree whenever the model will actually be served.

Q1.7 What are emergent abilities, and how would you respond to a colleague who claims they prove models will suddenly become dangerous at some scale?

The phenomenon. Wei et al. (2022) catalogued capabilities—multi-digit arithmetic, instruction following, chain-of-thought reasoning, in-context learning from few examples—that are at chance level in small models and rise sharply past a scale threshold, appearing discontinuous on a log-compute axis.

The strongest counter-argument. Schaeffer et al. (2023) showed that much of the apparent sharpness is a *measurement* artefact. Metrics like exact-match accuracy on a 5-

digit multiplication are all-or-nothing: a model that improves steadily from getting 1 digit right to 4 digits right scores zero the entire time, then jumps to one. Swap in a continuous metric—token-level edit distance, or log-likelihood of the correct answer—and the same runs show smooth, predictable improvement. The discontinuity was in the ruler, not the model.

How I would respond to the colleague. Carefully, in three parts.

First, the empirical claim is weaker than it looks. Under continuous metrics most "emergence" dissolves, so it is not evidence for an unpredictable capability cliff.

Second, the concern is not entirely dissolved either. Even if underlying capability improves smoothly, *downstream systems* often care about the discrete threshold. A model that goes from 40% to 85% on tool-use reliability crosses a line where a previously infeasible agentic product becomes feasible, and that transition is real regardless of what the loss curve looks like. Smooth capability growth can still produce discontinuous consequences.

Third, the actionable engineering conclusion is the same either way: do not extrapolate a capability you have not measured on your own task. Build an evaluation suite over your actual workload, run every candidate model against it, and let measured numbers—not scaling narratives in either direction—drive the decision.

Q1.8 Your RAG system returns semantically similar but factually wrong passages. A colleague proposes a better embedding model. Is that the right fix?

Probably not, and the reason is a property of the retrieval objective itself. Dense embeddings are trained with a contrastive objective to place *topically related* text nearby. Nothing in that objective encodes truth, negation, or temporal validity. Consider three passages:

- "Policy A-7 covers water damage from burst pipes." (current, correct)
- "Policy A-7 does not cover water damage from burst pipes." (negated)
- "Policy A-7 covered water damage until the 2023 revision." (stale)

All three sit within a few hundredths of cosine similarity of each other and of the query. A better encoder moves all three closer to the query together. It does not separate them, because separating them is not what it was trained to do.

The diagnosis to run first. Before changing anything, decompose the error. Measure recall@50 — does the correct passage appear anywhere in the candidate set?

- **Recall@50 is low** ⇒ a genuine retrieval failure. Now the embedding model, chunking strategy, or hybrid search *is* the fix.
- **Recall@50 is high but recall@5 is low** ⇒ a *ranking* failure. The right passage is being found and then out-ranked. A better bi-encoder will not fix this.

In the failure described, it is almost always the second case.

The actual fixes, in order of leverage.

1. **Cross-encoder reranking.** A cross-encoder attends jointly over query and passage, so it *can* represent "this passage negates the query premise." Retrieve 50, rerank to 5. This is the highest-leverage single change in most RAG systems.

2. **Metadata filtering before vector search.** Effective dates, document versions, and jurisdiction are structured fields. Filter on them; never hope the embedding encodes them. This alone eliminates the stale-passage class.
3. **Hybrid retrieval.** Combine BM25 with dense search via reciprocal rank fusion. Lexical search catches exact policy identifiers that dense retrieval blurs.
4. **Grounded generation with entailment verification.** Require the model to cite passage IDs, then score the answer against the cited passages with an NLI model. Contradictions surface as low entailment.
5. **Chunking review.** If a chunk boundary split "does not cover" from its subject, no retriever can recover the meaning. Check this before blaming the model.

The principle worth stating explicitly: retrieval optimises recall, reranking optimises precision, and generation optimises fluency. When the symptom is "plausible but wrong," the defect is nearly always in the precision stage, and upgrading the recall stage will not touch it.

Q1.9 Walk through what happens between a user pressing Enter and the first token appearing on screen.

Stage 1 — Request assembly (application, 1–50 ms). The application composes the message list: system prompt, conversation history, retrieved context, tool schemas, user message. Token counting happens here against a real budget. Anything dropped should be logged.

Stage 2 — Tokenization (sub-millisecond to a few ms). The string becomes integer IDs via BPE or SentencePiece. Chat models apply a chat template that inserts role delimiters—an easily-missed source of quality bugs when self-hosting, because using the wrong template silently degrades output.

Stage 3 — Admission and scheduling (variable). The request enters the serving engine's queue. A continuous-batching scheduler (vLLM, SGLang, TensorRT-LLM) decides which requests join the current step based on available KV-cache blocks. Under load, queueing—not compute—is the dominant term in tail latency.

Stage 4 — Prefill (compute-bound; the bulk of TTFT). All prompt tokens are processed in one parallel forward pass. Every layer computes Q, K, V; K and V are written to the KV cache. This phase is arithmetic-intensity-high and saturates the GPU's tensor cores. Cost grows superlinearly in prompt length. If a prefix cache hit occurs—a shared system prompt already resident—this phase can be skipped almost entirely, which is why prefix caching is the single most effective TTFT optimisation available (Section 4.5).

Stage 5 — First sampling step. The final hidden state is projected to a logit vector of size $|V|$. Temperature scaling, top-k, and top-p filtering shape the distribution; one token is sampled.

Stage 6 — Detokenization and streaming. The token ID becomes text. Because BPE tokens can be partial UTF-8 sequences, the server buffers incomplete codepoints rather than emitting broken characters. The chunk is pushed over SSE or a WebSocket.

Stage 7 — Decode loop. Each subsequent token requires a full forward pass over one token, reading the entire KV cache and all model weights from HBM. This phase is *memory-bandwidth-bound*, not compute-bound—the arithmetic intensity is roughly 1 FLOP per byte

loaded. This asymmetry between prefill and decode is the central fact of LLM serving and the reason batching helps decode enormously and prefill much less.

The number that matters. TTFT ≈ queue time + prefill time. End-to-end latency ≈ TTFT + (output tokens × time-per-output-token). Optimising the wrong one of those two is the most common serving mistake: teams tune decode throughput when 90% of their latency was a 12k-token prefill that a prefix cache would have eliminated.

Q1.10 Design the model-selection strategy for a startup with three workloads: high-volume classification, customer-facing chat, and a low-volume legal analysis tool.

The framing. There is no single right model—there is a right model *per route*, chosen by matching each workload's binding constraint to the axis on which models differ.

Workload	Binding constraint	Choice and rationale
Classification 50M/month	Unit cost, p99 latency	Fine-tuned encoder (ModernBERT-class, ≈150M params) on CPU or a single small GPU. This is not a generative task. Bootstrap labels with a frontier LLM once, then distil.
Chat 2M/month	Quality × latency, brand risk	Mid-tier hosted model with streaming, plus a cheaper fallback tier for simple turns. Route by an intent classifier, not by prompt length.
Legal analysis 5k/month	Accuracy, auditability	Frontier long-context model. At 5k requests a month, cost is irrelevant and a single bad analysis is expensive. Add citation requirements and human review.

The engineering that makes this work.

1. **A provider-agnostic gateway.** Every call goes through one internal interface. Model choice is configuration, not code. This is what makes the routing table changeable in an afternoon when prices move.
2. **An evaluation suite per route, built before the first model is chosen.** 100–300 examples with graded rubrics. Without this you are choosing models by vibes and you cannot detect regression when a vendor silently updates a checkpoint.
3. **Per-route observability.** Track tokens in/out, latency percentiles, cost, and quality score by route. The routing decision should be revisited quarterly against measurements.
4. **Explicit fallback chains.** Primary → secondary provider → degraded deterministic path. Provider outages are a normal operating condition, not an exception.

```
A routing layer that makes model choice configuration, not code
```

```python
from dataclasses import dataclass

@dataclass(frozen=True)
class Route:
    name: str
    primary: str
    fallback: str | None
    max_output: int
    temperature: float
    monthly_budget_usd: float

ROUTES = {
    "classify": Route("classify", "local/modernbert-intent", None,
                      max_output=8,    temperature=0.0, monthly_budget_usd=400),
    "chat":     Route("chat", "vendor-a/mid-tier", "vendor-b/mid-tier",
                      max_output=1024, temperature=0.7, monthly_budget_usd=6_000
),
    "legal":    Route("legal", "vendor-a/frontier-long", "vendor-b/frontier",
                      max_output=8192, temperature=0.2, monthly_budget_usd=900),
}

def complete(route_name: str, messages: list[dict]) -> str:
    r = ROUTES[route_name]
    for model in filter(None, (r.primary, r.fallback)):
        try:
            resp = gateway.call(model, messages,
                                max_tokens=r.max_output,
                                temperature=r.temperature)
            metrics.record(route=r.name, model=model,
                           tokens=resp.usage, latency=resp.latency_ms)
            return resp.text
        except (ProviderError, TimeoutError) as e:
            metrics.increment("route.fallback", route=r.name,
            ↪  cause=type(e).__name__)
    raise AllProvidersFailed(route_name)
```

The trap to avoid. Startups routinely standardise on one frontier model "for simplicity" and discover at scale that 80% of their spend is on requests a 1/50-cost model would have handled identically. Build the gateway on day one even if every route points at the same model—the abstraction costs a day and buys you the ability to act on that discovery when it arrives.

References

Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., von Arx, S., … Liang, P. (2021). *On the opportunities and risks of foundation models* (arXiv:2108.07258). arXiv. https://arxiv.org/abs/2108.07258

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 770–778). https://doi.org/10.1109/CVPR.2016.90

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., … Sifre, L. (2022).

Training compute-optimal large language models (arXiv:2203.15556). arXiv. https://arxiv.org/abs/2203.15556

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., ... Amodei, D. (2020). *Scaling laws for neural language models* (arXiv:2001.08361). arXiv. https://arxiv.org/abs/2001.08361

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics, 12*, 157–173. https://doi.org/10.1162/tacl_a_00638

Schaeffer, R., Miranda, B., & Koyejo, S. (2023). Are emergent abilities of large language models a mirage? In *Advances in Neural Information Processing Systems* (Vol. 36). https://arxiv.org/abs/2304.15004

Sennrich, R., Haddow, B., & Birch, A. (2016). Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics* (pp. 1715–1725). https://doi.org/10.18653/v1/P16-1162

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems* (Vol. 30). https://arxiv.org/abs/1706.03762

Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., ... Fedus, W. (2022). Emergent abilities of large language models. *Transactions on Machine Learning Research*. https://arxiv.org/abs/2206.07682

Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., ... Liu, T.-Y. (2020). On layer normalization in the Transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning* (pp. 10524–10533). https://arxiv.org/abs/2002.04745

Zhang, B., & Sennrich, R. (2019). Root mean square layer normalization. In *Advances in Neural Information Processing Systems* (Vol. 32). https://arxiv.org/abs/1910.07467

CHAPTER 2

Transformer and LLM Architecture

Subscribe → aiengineeringinsider.substack.com/subscribe

Every architectural choice in a modern LLM—RoPE, GQA, MoE, FlashAttention—is an answer to a memory or bandwidth problem, not an intelligence problem. This chapter teaches you to read architecture as systems engineering.

2.1 Overview

The 2017 Transformer and a 2026 frontier model share a skeleton and almost nothing else. The differences are not incidental: each one was introduced to break a specific scaling wall, and every one of them shows up as a line item in your serving bill.

Component	2017 original	2026 production default, and the wall it broke
Position	Sinusoidal, additive	RoPE — relative, extrapolates, no learned table to resize
Normalisation	Post-LN, LayerNorm	Pre-LN, RMSNorm — trains at 80+ layers without warmup tuning
Activation	ReLU	SwiGLU — better quality per parameter
Attention	Multi-head (MHA)	GQA or MLA — shrinks the KV cache 4–8×
Attention kernel	Materialised $T \times T$ matrix	FlashAttention — $O(T)$ memory, makes 128k context possible
FFN	Dense, always active	MoE — 10× parameters at constant per-token FLOPs
Bias terms	Present	Removed — no quality cost, fewer parameters and kernels

⋆ Key Insight

Read that table as a single story: after 2020 the binding constraint on language models stopped being FLOPs and became **memory bandwidth**. GQA, MLA, MoE, FlashAttention, and quantization are all bandwidth optimisations wearing different clothes. If you can articulate that in an interview, you have shown that you understand *why* the architecture looks the way it does rather than merely what it contains.

2.2 The Transformer Block

A modern decoder-only block is two residual sub-layers:

$$h = x + \text{Attn}(\text{RMSNorm}(x))$$
$$y = h + \text{FFN}(\text{RMSNorm}(h))$$

The **residual stream** is the tensor flowing down the left-hand side, untouched by normalisation. Each block *reads* from it, computes, and *writes back* an increment. This framing—due to the mechanistic interpretability literature—is the most useful mental model available: layers communicate through a shared bus, and attention heads at layer 20 can read what a head at layer 3 wrote.

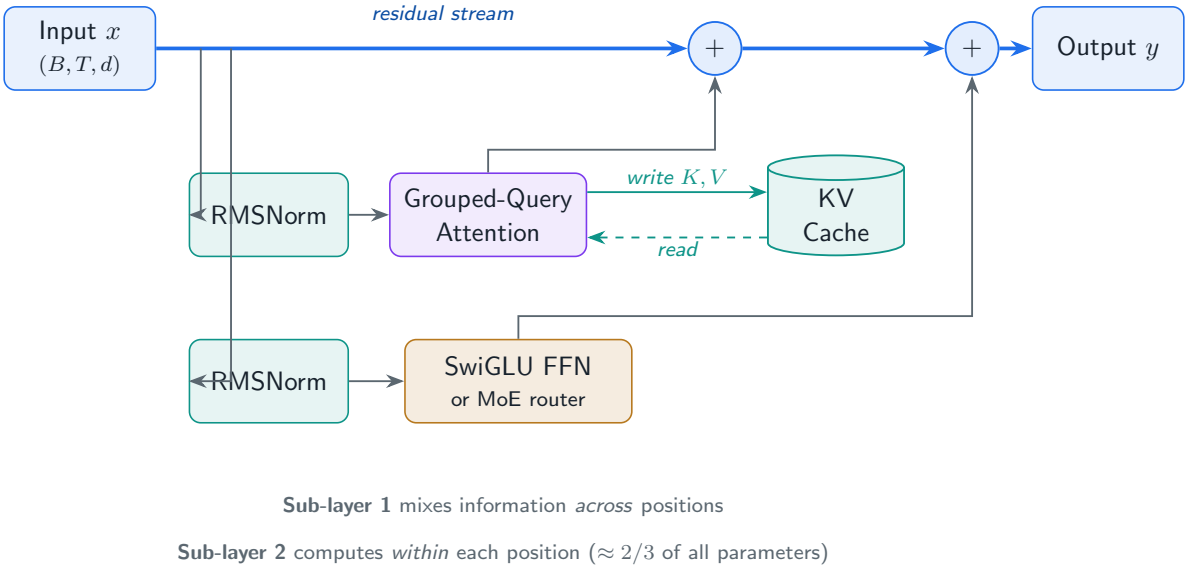


Figure 2.1: A modern decoder-only Transformer block. The residual stream runs straight through and is never normalised; each sub-layer normalises only its own branch input. Note where the KV cache is written and read—that is the memory that dominates serving.

2.3 Attention Variants

2.3.1 Self, Cross, and Causal Attention

Variant	Q, K, V come from	Where it appears
Bidirectional self-attention	Same sequence, no mask	BERT-style encoders; embedding and reranking models
Causal self-attention	Same sequence, upper-triangular mask	Every decoder-only LLM; the mask is what enables KV caching
Cross-attention	Q from decoder, K/V from encoder	T5, Whisper, Flamingo-style VLM connectors

2.3.2 MHA, MQA, GQA, and MLA

This progression is the most reliably asked architecture question in inference interviews, because it is pure systems engineering: four ways to trade quality against KV-cache size.

MHA Multi-Head Attention. h query heads, h key heads, h value heads. Maximum expressivity, maximum cache.

MQA Multi-Query Attention (Shazeer, 2019). h query heads share *one* K/V head. Cache

shrinks by $h\times$; quality degrades measurably on hard tasks.

GQA Grouped-Query Attention (Ainslie et al., 2023). h query heads share g K/V heads, $1 < g < h$. Typically $g = 8$. Recovers nearly all MHA quality at h/g times less cache. The production default.

MLA Multi-head Latent Attention (DeepSeek-AI, 2024). Compresses K and V into a low-rank latent vector that is cached instead; K and V are reconstructed on the fly. Largest reduction, most implementation complexity.

Scheme	KV heads	Cache per token, 70B-class	Trade-off
MHA	64	≈ 2.5 MB	Best quality; cache dominates memory and caps batch size
MQA	1	≈ 0.04 MB	64× smaller; noticeable quality loss, training instability
GQA-8	8	≈ 0.31 MB	8× smaller, quality within noise of MHA. Default choice
MLA	latent	≈ 0.07 MB	Largest reduction with strong quality; complex kernels

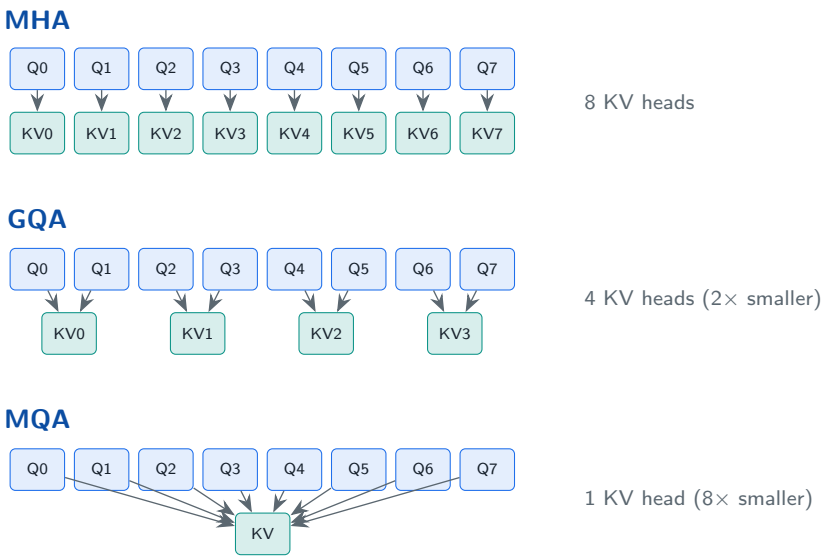


Figure 2.2: MHA, GQA, and MQA differ only in how many query heads share a key/value head. The shared K/V heads are what you pay for in cache memory.

2.4 Positional Encoding

Attention is permutation-equivariant: shuffle the input tokens and you get the same outputs, shuffled. Position must be injected explicitly.

Method	Mechanism	Extrapolation and notes	Used by
Sinusoidal	Fixed sin/cos added to embeddings	Poor beyond training length; absolute, not relative	Original Transformer
Learned absolute	Trainable table of size $T_{\max}$	None—hard cap at table size; resizing needs retraining	BERT, GPT-2
RoPE	Rotate Q/K by angle $\propto$ position	Encodes *relative* distance in the dot product; extends via base scaling (NTK, YaRN)	Llama, Qwen, Mistral, DeepSeek
ALiBi	Linear distance penalty added to scores	Extrapolates well by construction; no vectors to store	BLOOM, MPT
M-RoPE	RoPE split across time/height/width axes	Encodes 2D and temporal position for images and video	Qwen-VL family

2.4.1 Why RoPE Won

Rotary Position Embedding (Su et al., 2024) rotates the query and key vectors in 2D subspaces by an angle proportional to absolute position. The elegance is that the *dot product* of a rotated query at position m and a rotated key at position n depends only on $m - n$:

$$\langle R_m q,\ R_n k \rangle \ = \ g(q, k, m - n).$$

So RoPE encodes absolute position at write time and produces relative position at read time. Three practical consequences:

1. **It is applied to Q and K only, never V.** Position should influence *which* tokens you attend to, not *what content* they carry.
2. **It is compatible with KV caching.** Rotation is applied to K before it is written to cache, and never needs revisiting.
3. **Context can be extended post-hoc.** Scaling the RoPE base frequency (NTK-aware scaling) or interpolating positions (YaRN; Peng et al., 2024) extends usable context well past the training length with modest continued pretraining.

RoPE applied to Q and K, with a base-scaling knob for context extension

```
import torch

def build_rope_cache(seq_len, head_dim, base=10_000.0, scale=1.0, device="cuda"):
    """scale > 1 stretches the frequency spectrum (NTK-aware context extension)."""
    inv_freq = 1.0 / ((base * scale) ** (torch.arange(0, head_dim, 2, device=device)
    ↪  / head_dim))
    t = torch.arange(seq_len, device=device, dtype=inv_freq.dtype)
    freqs = torch.outer(t, inv_freq)              # (T, head_dim/2)
    return freqs.cos(), freqs.sin()               # each (T, head_dim/2)

def apply_rope(x, cos, sin):
```

```
    """x: (B, H, T, head_dim). Rotates adjacent dimension pairs."""
    x1, x2 = x[..., 0::2], x[..., 1::2]
    cos, sin = cos[None, None, :, :], sin[None, None, :, :]
    out = torch.stack([x1 * cos - x2 * sin,
                       x1 * sin + x2 * cos], dim=-1)
    return out.flatten(-2)

# In the attention forward pass -- note V is untouched:
cos, sin = build_rope_cache(T, head_dim, scale=4.0)   # 4x context extension
q = apply_rope(q, cos, sin)
k = apply_rope(k, cos, sin)
# k is now written to the KV cache already rotated; it is never re-rotated.
```

! Pitfall / Warning

Extending RoPE context by naively increasing the base frequency without continued pre-training produces a model that *accepts* 128k tokens and *reasons* over about 8k. The failure is silent: no error, no truncation, just degraded recall for anything past the original training length. Always validate an extended context with a needle-in-a-haystack sweep across depths before shipping it.

2.5 The KV Cache

This is the most important systems concept in LLM serving, and it deserves a first-principles derivation rather than a definition.

2.5.1 Why It Exists

Generating token $t+1$ requires attention over all previous tokens—which requires their K and V vectors. Those vectors depend only on tokens $1..t$, which have not changed. Recomputing them each step would make generating n tokens cost $O(n^2)$ forward passes over the whole prefix. Caching them makes it $O(n)$.

KV Cache

The per-layer, per-head store of key and value tensors for every token processed so far. It converts autoregressive generation from quadratic recomputation into linear incremental work, at the cost of memory that grows linearly with sequence length *and* batch size.

2.5.2 Sizing It—the Formula You Must Know

$$\text{KV bytes} = 2 \times L \times H_{kv} \times d_{\text{head}} \times T \times B \times \text{bytes_per_element}$$

where the leading 2 counts K and V, L is layers, H_{kv} is *key/value* heads (not query heads—this is where GQA pays off), T is sequence length, and B is batch size.

`KV-cache sizing: the calculation that determines your max batch size`